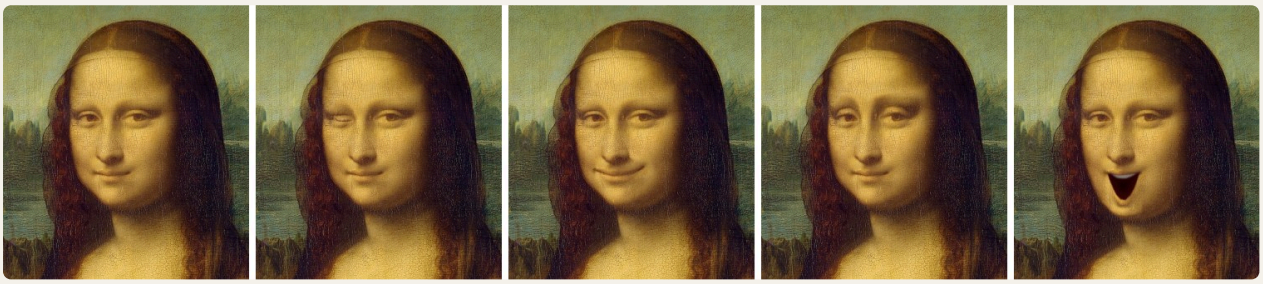
PROJET DE VISION PAR ORDINATEUR

Portrait vivant

Le portrait qui imite vos grimaces en direct : dossier de compréhension et de préparation à la soutenance



De gauche à droite : le portrait au repos, puis un clin d'œil, un sourire, des sourcils levés et une bouche ouverte, calculés par le programme à partir de grimaces simulées.

Master 2, Vision par ordinateur · IMT Nord Europe
Version du 6 novembre 2026

Sommaire

1. Le projet en une page 3

2. Le contexte et les objectifs 4

3. L'idée générale et le fonctionnement 5

4. Le programme, étape par étape 7

Repères · Portrait · Triangles · Déformation · Lissage · Recalage · Transfert · Yeux · Rotation · Garde-fous · Bouche · Interface

5. Le lien avec le cours 15

6. Les résultats et leur lecture 16

7. Les problèmes rencontrés et nos décisions 19

8. Les limites à assumer 21

9. Préparer la présentation et la démo 22

10. Les questions probables du jury 24

Annexes : code, paramètres, commandes et lexique 29

Comment utiliser ce dossier

Pour comprendre le projet, lisez les sections 1 à 4 dans l'ordre. Avant la soutenance, révisez les encadrés « À retenir » et « À dire à l'oral », puis entraînez-vous à répondre aux questions de la section 10 avec vos propres mots.

Ce qui reste à vérifier de votre côté

Les illustrations de ce dossier viennent du vrai code, mais les grimaces y sont simulées, sauf indication contraire. L'interface web n'a été testée qu'avec une fausse caméra. Les mesures datent d'avant l'interface et la nouvelle bouche : relancez-les sur la version finale.

1. Le projet en une page

Portrait vivant fait refaire vos grimaces à n'importe quel portrait, en direct, à partir d'une seule image et d'une webcam. Vous clignez de l'œil devant la caméra, et la Joconde cligne de l'œil à côté de vous. Tout tourne sur un portable ordinaire, sans carte graphique et sans service en ligne.

24	117	236	≈ 5×
images par seconde mesurées, sans GPU	repères suivis sur le visage, à chaque image	triangles, donc 236 transformations affines par image	moins d'erreur en 3D qu'en 2D quand la tête tourne (2,9 px contre 14 px)

> **L'idée à retenir, en une phrase**
> On ne copie pas le visage filmé : on mesure comment ses points bougent par rapport à son visage au repos, on déplace les mêmes points sur le portrait, puis on déforme l'image triangle par triangle.

Ce que le programme reçoit et produit

- **Entrée 1 :** une seule image de portrait, de préférence de face, yeux ouverts et bouche fermée.
- **Entrée 2 :** le flux de la webcam.
- **Sortie :** vous à gauche, le portrait animé à droite, mis à jour à chaque image.

Les trois difficultés du projet

1. **Séparer la tête de la grimace.** Quand je bouge la tête, tous mes points bougent, sans que je grimace. C'est le rôle du recalage, et c'est notre résultat le plus intéressant.
2. **Déformer sans déchirer.** Il faut déplacer des pixels sans trous, sans plis et assez vite pour le direct : c'est le rôle des triangles et des transformations affines.
3. **Montrer ce qui n'existe pas.** Le portrait a la bouche fermée : il n'y a aucun pixel de l'intérieur de sa bouche. Il faut donc le peindre.



Une même grimace sur six tableaux. Le programme s'adapte à chaque visage, à sa taille et à sa couleur. Grimace simulée. Sur Washington, la lèvre garde une forme en créneau : le résultat dépend du portrait.

2. Le contexte et les objectifs

Ce projet a été réalisé par trois étudiants de Master 2 en vision par ordinateur, à l'IMT Nord Europe, en une semaine. Il devait mobiliser les notions de trois cours : l'alignement d'images, la projection et calibration de caméra, et l'initiation à l'image numérique.

Le cahier des charges

Contrainte	Conséquence pour le projet
Tout en local, sans GPU	Méthode géométrique, assez rapide pour un processeur de portable
Une seule image du portrait	Aucune vidéo préparée : on ne peut déformer que les pixels présents dans cette image
Une seule caméra	Pas de stéréo ni de géométrie épipolaire
Une semaine de travail	Pas de NeRF ni de 3DGS, trop lourds. Un programme simple et expliqué avant tout
Python, avec MediaPipe et OpenCV	La détection des repères est déléguée à MediaPipe ; nous programmons toute la géométrie qui suit
Respect des personnes	Œuvres du domaine public, ou photos de personnes qui ont donné leur accord

Pourquoi cette méthode, et pas une IA générative ?

Des modèles comme LivePortrait peuvent produire un résultat plus réaliste en générant des pixels. Ils demandent toutefois plus de calcul, et leur fonctionnement interne est plus difficile à expliquer. Notre méthode a l'avantage inverse : **chaque pixel déplacé s'explique par une formule du cours**. C'est bien l'objectif pédagogique du projet.

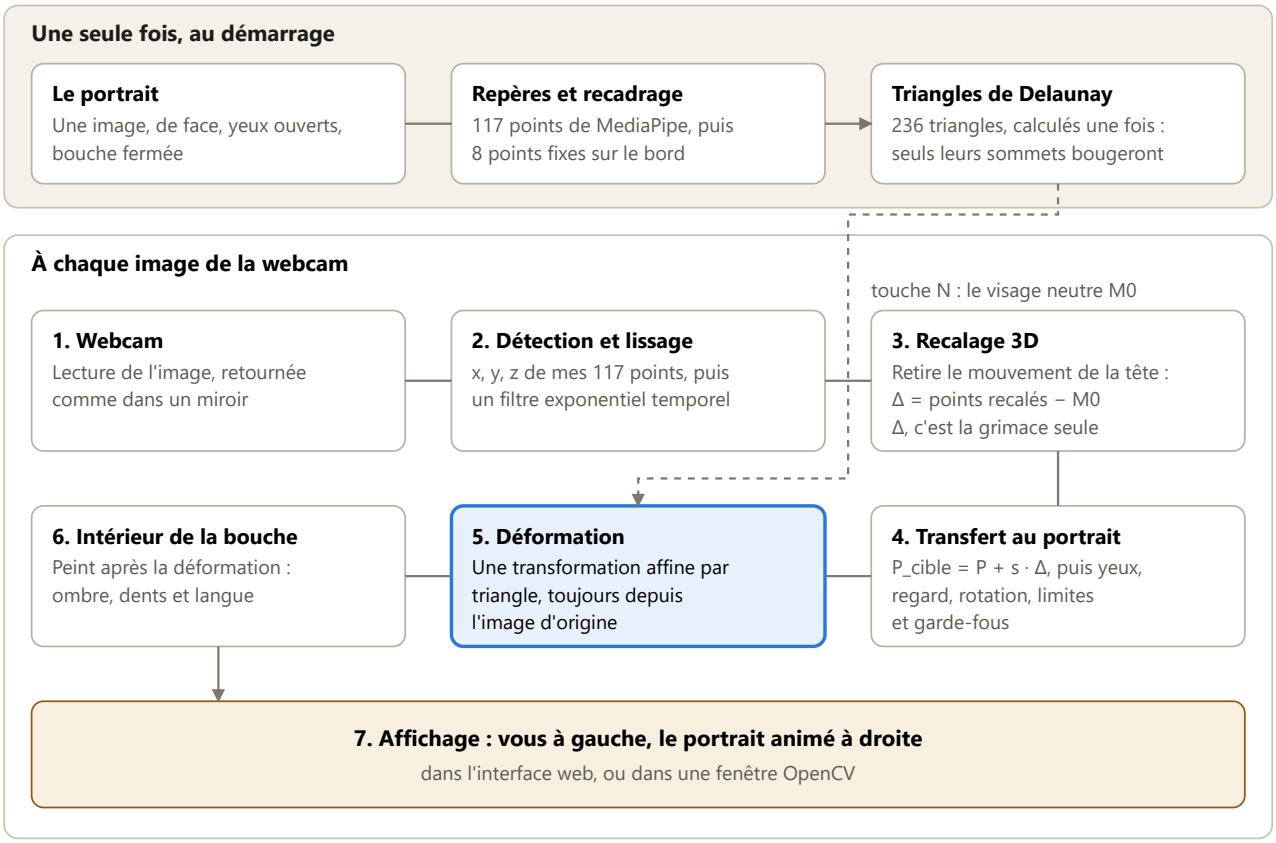
La comparaison avec LivePortrait était un bonus du sujet. Elle n'a pas été réalisée : n'annoncez donc aucun chiffre ni résultat à son sujet.

Les outils utilisés

Outil	Version	Rôle
Python	3.14.7	Langage du projet
MediaPipe (Face Landmarker)	1.2.2	Détecter 478 repères du visage, avec x, y et une profondeur z
OpenCV	5.0.0	Webcam, transformations affines, masques, filtres, images JPEG
NumPy et SciPy	2.5.4 et 1.18.1	Calcul matriciel, décomposition en valeurs singulières, triangulation de Delaunay
Matplotlib et pytest	3.11.2 et 9.2.0	Graphiques du rapport et tests automatiques

3. L'idée générale et le fonctionnement

Le programme prépare le portrait une seule fois, puis répète une même chaîne d'étapes à chaque image de la webcam. Les triangles calculés au démarrage ne changent jamais : seule la position de leurs sommets change.



Le fonctionnement complet. La flèche en pointillé montre que la déformation réutilise toujours les mêmes triangles. L'étape bleue est le cœur du projet et le lien principal avec le cours.

Pourquoi mesurer un déplacement ?

Mon visage et celui du portrait n'ont pas la même forme. Si je copiais la position de mes points sur le portrait, il prendrait la forme de ma tête. On calcule donc Δ, le déplacement de chaque point par rapport à mon visage au repos, et on l'ajoute aux points du portrait. Le portrait garde sa forme et reçoit seulement ma grimace.

$$P_{\text{cible}} = P_{\text{portrait}} + s \cdot \Delta \quad \text{avec} \quad \Delta = M_{t,\,\text{recalé}} - M_0$$

M_0 : mon visage au repos · M_t : mes points à l'instant t · s : rapport de taille entre le portrait et moi

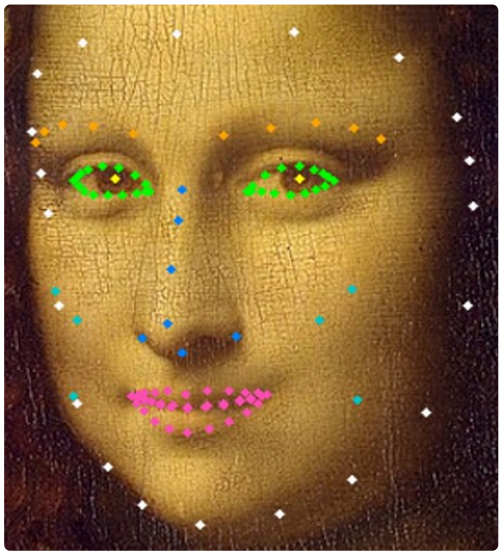
Un exemple complet : le clin d'œil

1. Je regarde la caméra, visage détendu, et j'appuie sur N : le programme retient la position de mes points.
2. Je ferme l'œil droit : les points de ma paupière haute descendent.
3. Si ma tête a bougé en même temps, le recalage retire ce mouvement. Il ne reste que le geste de la paupière.
4. Le programme calcule que mon œil n'est plus ouvert qu'à une petite part de son ouverture au repos.
5. Il applique cette proportion à l'œil du portrait, quelle que soit sa taille.
6. Les sommets des triangles autour de l'œil se déplacent. Chaque triangle reçoit sa transformation affine.
7. Le portrait ferme l'œil, du même côté de l'écran que moi, grâce à l'effet miroir.

Les grandeurs à ne pas confondre

Presque toutes les explications du projet reposent sur ces six notations. Les connaître évite la plupart des confusions à l'oral.

Notation	Ce que c'est	À qui ça appartient	Quand c'est calculé
$\mathbf{M_0}$	Mes points au repos : le visage neutre	À moi	Une fois, avec la touche N
$\mathbf{M_t}$	Mes points à l'instant t, lissés	À moi	À chaque image
$\mathbf{\Delta}$	M_t recalé − M_0 : la grimace, sans la tête	À moi, puis transférée	À chaque image
s	Le rapport de taille entre le portrait et moi	Au couple portrait-personne	À chaque image, à partir de M_0
P	Les points du portrait au repos	Au portrait	Une fois, au chargement
$\mathbf{P_{cible}}$	Les points du portrait qui grimace	Au portrait	À chaque image



Les 117 repères sur le visage du portrait. Blanc : contour · orange : sourcils · vert : paupières · jaune : centre des iris · bleu : nez · rose : lèvres · turquoise : joues.

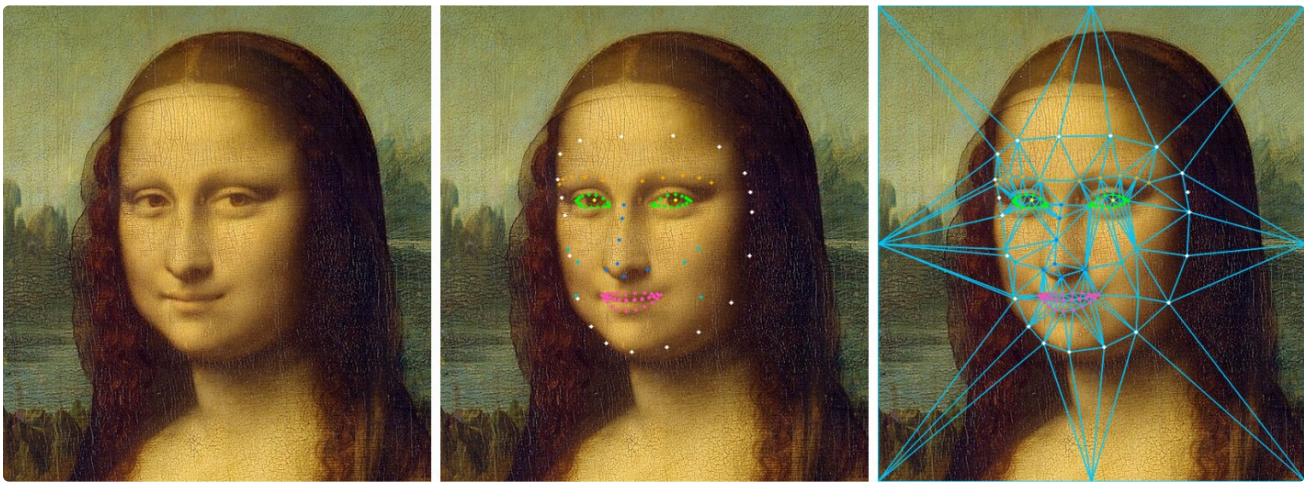
4. Le programme, étape par étape

4.1 Détecter les repères du visage

MediaPipe donne 478 points du visage à chaque image ; nous en gardons 117. Ce sous-ensemble suffit pour décrire une expression, et chaque point supplémentaire ajoute des triangles à calculer. Les indices ne sont pas écrits au hasard : ils viennent des listes de connexions fournies par MediaPipe, et un test automatique le vérifie.

Partie du visage	Points	Pourquoi
Lèvres, extérieur et intérieur	40	La zone la plus expressive, qui a besoin de deux contours
Paupières (les deux yeux)	32	Pour fermer l'œil précisément
Contour du visage et tempes	20	La silhouette. Les tempes servent au recalage, pas aux triangles
Sourcils (bord supérieur)	10	Pour les lever et les froncer
Nez	7	Un point stable pour le recalage, et du relief pour la rotation
Joues	6	Sans eux, le sourire se perdait dans de très grands triangles
Centre des iris	2	Pour suivre le regard

MediaPipe utilise un réseau de neurones entraîné : c'est la seule partie du programme que nous n'avons pas écrite. Elle donne aussi une profondeur z pour chaque point, estimée et non mesurée, qui sera essentielle au recalage.



Du portrait aux triangles. L'image d'origine recadrée, ses repères par partie du visage, puis la triangulation complète, avec les 8 points fixes du bord.

4.2 Préparer le portrait

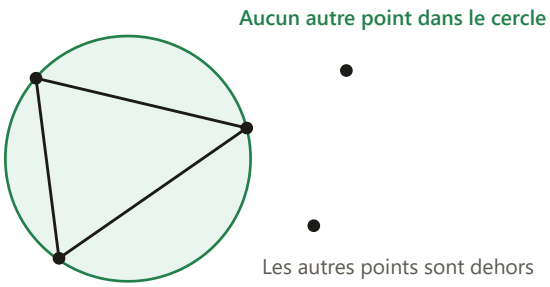
Cette préparation n'a lieu qu'une fois par portrait, dans `preparer_portrait` :

1. **Recadrage :** l'image est coupée autour du visage avec une marge de 60 % de sa taille. Sur la Joconde entière, le visage serait trop petit pour que les grimaces se voient.
2. **Mise à l'échelle :** l'image passe à 600 pixels de haut, pour garder une déformation rapide et un affichage adapté à un portable.

3. **Points de bord :** 8 points fixes (les 4 coins et les 4 milieux) gardent le cadre immobile. Sans eux, les bords de l'image seraient déformés.
4. **Triangulation :** elle est calculée une fois pour toutes (section suivante).
5. **Couleurs :** la couleur de l'intérieur de la bouche et celle des dents sont tirées du tableau.

4.3 Découper en triangles : Delaunay

La triangulation de Delaunay relie les points en évitant autant que possible les triangles très fins. Sa propriété : le cercle passant par les 3 sommets d'un triangle ne contient aucun autre point. Avec `scipy.spatial.Delaunay`, on obtient 236 triangles sur la Joconde.



La propriété de Delaunay. Un cercle circonscrit vide écarte les triangles trop fins.

Quelques points retirés des sommets

Sur un visage de trois quarts, le bord de la joue éloignée se projette parfois sur l'œil. Sur Vermeer, la tempe tombe même à l'intérieur de l'œil, ce qui plierait l'image à chaque clignement. Les tempes et tout point du contour tombant dans un œil ne deviennent donc pas des sommets.

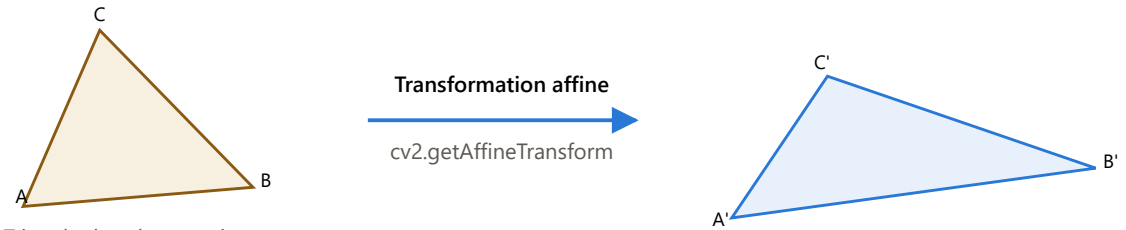
Un ordre de dessin réfléchi : les triangles entièrement dans un œil ou entre les lèvres sont dessinés en premier. Ils se replient forcément quand l'œil se ferme, mais la peau dessinée ensuite les recouvre, donc ces replis restent invisibles.

4.4 Déformer : le cœur du cours

Chaque triangle reçoit sa propre transformation affine, calculée à partir de ses 3 sommets avant et après. Une transformation affine a 6 inconnues ; chacun des 3 points donne 2 équations. Il y a donc exactement une solution.

$$x' = a \cdot x + b \cdot y + t_x \qquad y' = c \cdot x + d \cdot y + t_y$$

6 inconnues : a, b, c, d, t_x, t_y · 3 points × 2 coordonnées = 6 équations



Triangle dans le portrait — Triangle après déplacement des points

Trois points suffisent. Les droites restent droites, et le côté partagé par deux triangles voisins reste le même : l'image n'a donc pas de trou.

Pour chaque triangle, le programme calcule la petite boîte qui l'englobe, déforme seulement cette zone, crée un masque de la forme du triangle, puis recolle le résultat. Travailler sur la boîte évite de déformer les 600 pixels de haut 236 fois.

```
# deformation.py : l'essentiel de deformer_triangle (version simplifiée)
affine = cv2.getAffineTransform(t1, t2)              # 3 points → 3 points : 6 inconnues
morceau = cv2.warpAffine(src[ys0:ys1, xs0:xs1], affine, (x1 - x0, y1 - y0),
                         flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE)
masque = np.zeros((y1 - y0, x1 - x0), np.uint8)
cv2.fillConvexPoly(masque, np.int32(np.round(t2)), 255)  # la forme du triangle
cv2.copyTo(morceau, masque, dst[y0:y1, x0:x1])       # on ne recolle que l'intérieur
```

> **À dire à l'oral**
> « Une transformation affine par triangle : chaque triangle se déforme selon une formule simple, et comme deux triangles voisins partagent leurs sommets, l'image reste continue. Nous repartons toujours de l'image d'origine pour ne pas accumuler de flou. »

Trois détails à connaître

- **Toujours depuis l'original :** chaque interpolation floute un peu. Repartir du résultat précédent accumulerait ce flou.
- **Masque net :** notre fonction de départ demandait un masque anticrénelé, mais OpenCV ignore `LINE_AA` sur un masque `float32`. Un masque 8 bits net ne laisse aucun joint, car deux triangles voisins se recouvrent sur leur bord. Il a été 2,7 fois plus rapide dans notre essai (28 ms contre 77 ms, à 720 pixels de haut).
- **Vérification :** sans déplacement, l'image reconstruite sur fond noir est identique à l'originale. Un test le vérifie, et un autre contrôle qu'une vraie déformation ne laisse aucun trou.

4.5 Le visage neutre et le lissage

La touche N enregistre vos repères comme référence : le visage neutre M_0. Sans cette référence, on ne saurait pas ce qui a changé. Faites-la visage détendu : une grimace à cet instant fausserait toutes les suivantes.

Les repères tremblent légèrement d'une image à l'autre. Un **filtre exponentiel** fait une moyenne pondérée entre la nouvelle mesure et l'état précédent :

$$L_t = \alpha \cdot M_t + (1 - \alpha) \cdot L_{t-1}$$

α = 0,5 pour les repères · α = 0,25 pour les angles de la tête, car ils déplacent tout le visage

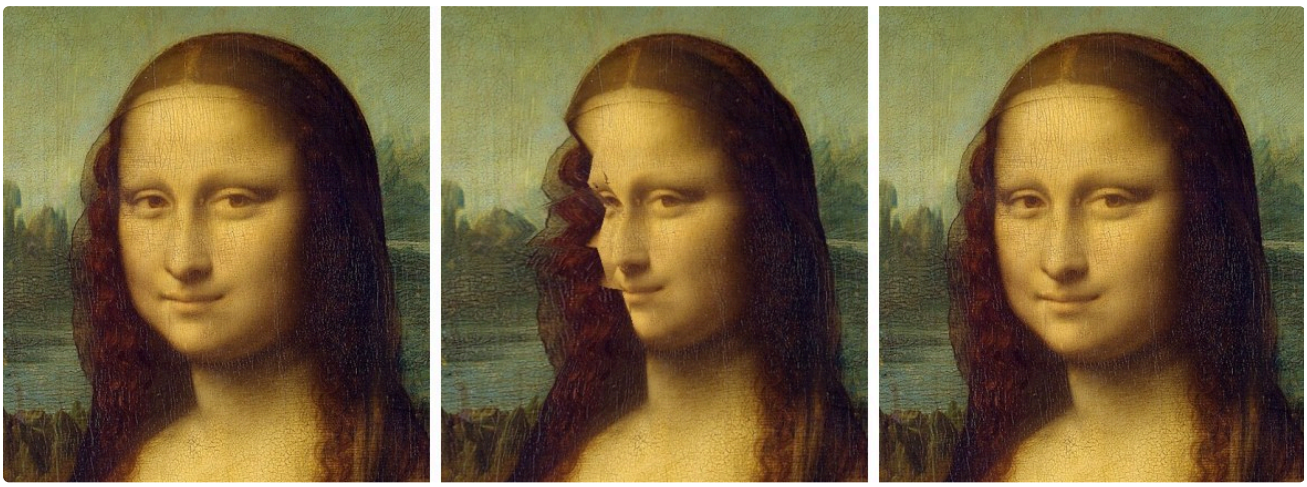
C'est un compromis : plus α est petit, plus le visage est stable, mais plus il suit avec retard. Si le visage disparaît, le filtre repart de zéro au retour, et le portrait garde son dernier état entre-temps.

4.6 Le recalage : retirer le mouvement de la tête

Le recalage remet ma tête dans sa position de référence, pour que seule la grimace reste dans Δ. On utilise des points qui bougent peu quand on grimace : coins des yeux, haut et arête du nez, tempes et haut du front, soit 9 points.

Première version : la similitude 2D

Notre sujet demandait une similitude 2D (rotation, échelle, translation : 4 inconnues), estimée par `cv2.estimateAffinePartial2D` avec RANSAC. Elle corrige bien une tête qui se déplace, s'approche ou penche. Mais tourner la tête est une **rotation en 3D** : le nez, plus proche, bouge plus que les joues. Aucune similitude 2D ne peut compenser cela, et le programme prend alors le mouvement de la tête pour une grimace.



La preuve par l'image, pour une tête tournée de 25° sans aucune grimace. À gauche, le portrait au repos. Au centre, le recalage 2D : Δ vaut en moyenne 19,8 px et déchire le visage. À droite, le recalage 3D : Δ vaut 0 px. La rotation est simulée et parfaite par construction : avec une vraie tête, il reste un petit écart (voir section 6).

Version retenue : la similitude 3D

Comme MediaPipe donne aussi z, on peut chercher une rotation R en 3D, avec une échelle s et une translation t, par moindres carrés :

$$\min_{s,\,R,\,t} \ \Sigma_i \,\| \, s \cdot R \cdot p_i + t - q_i \, \|^2$$

p_i : mes points stables à l'instant t · q_i : les mêmes points sur le visage neutre

Ce problème a une solution exacte, la **méthode d'Umeyama**. En voici le principe, à savoir expliquer :

1. **Centrer :** on soustrait à chaque nuage de points son centre, ce qui élimine la translation.
2. **Trouver R :** la décomposition en valeurs singulières (SVD) de la matrice de covariance des deux nuages donne la meilleure rotation. Un signe empêche d'obtenir une symétrie miroir au lieu d'une rotation.
3. **Trouver s puis t :** ils se déduisent directement de R et des centres.
4. **Écarter les aberrants :** les points trop éloignés du résultat sont retirés, puis on recommence une fois.

```
# recalage.py : similitude_3d_moindres_carres
centre_source, centre_cible = source.mean(axis=0), cible.mean(axis=0)
s_centre, c_centre = source - centre_source, cible - centre_cible
u, valeurs, vt = np.linalg.svd(c_centre.T @ s_centre)
signes = np.array([1.0, 1.0, np.sign(np.linalg.det(u @ vt))])  # une rotation, pas un miroir
rotation = u @ np.diag(signes) @ vt
echelle = (valeurs * signes).sum() / (s_centre**2).sum()
return echelle, rotation, centre_cible - echelle * rotation @ centre_source
```

> **RANSAC : soyez précis**
> RANSAC tire des points au hasard, en déduit un modèle, compte les points qu'il explique, puis garde le meilleur tirage. Notre recalage 3D ne fait **pas** cela : il rejette simplement les points éloignés d'une première estimation. Le vrai RANSAC est seulement dans le recalage 2D, gardé pour la comparaison. Ne dites donc pas que le programme principal utilise RANSAC.

Le z de MediaPipe est-il fiable ? Nous l'avons testé sur de vrais visages avant de nous y fier. Entre portraits orientés différemment de 15 à 19°, l'erreur passe de 6-13 % de l'écart entre les yeux en 2D à 2-4 % en 3D.

4.7 Transférer au portrait

Δ est mesuré sur mon visage, en pixels de ma webcam. Il faut l'adapter :

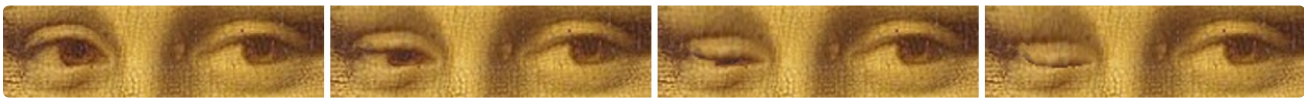
- **Échelle :** s = écart entre les yeux du portrait ÷ écart entre les miens.
- **Gains :** sur un tableau, un mouvement fidèle paraît trop discret. Les lèvres reçoivent un gain de 1,5, les joues et les sourcils de 1,3.
- **Intensité :** un réglage global de 0 à 2, avec le curseur ou les touches.
- **Limite :** un point ne peut pas se déplacer de plus de 0,8 fois l'écart entre les yeux du portrait.

4.8 Les yeux et le regard

On transfère une proportion d'ouverture plutôt que des pixels. En pixels, le clin d'œil restait faible : les yeux du portrait et les miens n'ont pas la même taille, et les points de MediaPipe ne se touchent pas tout à fait sur un œil fermé.

$$\text{ouverture relative} = \text{ouverture actuelle} \div \text{ouverture au repos}$$

En dessous de 0,35, l'œil compte comme fermé. Puis la paupière haute se place à : paupière basse + ouverture × écart au repos



L'œil droit à 100, 75, 55 et 30 % d'ouverture mesurée. La dernière valeur étant sous le seuil, l'œil se ferme entièrement. Simulation.

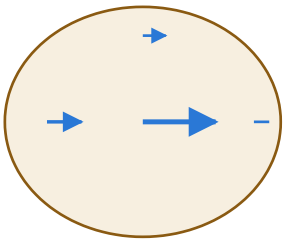
Le regard : l'iris bouge très peu dans l'œil, donc un transfert fidèle resterait invisible. On mesure sa position relative entre les coins de l'œil, on fait la moyenne des deux yeux pour éviter de faire loucher le portrait, on retire le bruit avec une zone morte (2 % de la largeur de l'œil), puis on amplifie par 2,5. La pupille reste entre 20 et 80 % de la largeur de l'œil, et entre les paupières quand il se ferme.



Le regard vers la gauche, au centre, puis vers la droite, sur Vermeer. Seule la pupille change. Simulation.

4.9 Faire tourner la tête du portrait

Le recalage retire la rotation de ma tête ; on la réapplique au portrait de façon contrôlée, avec un modèle de tête en ellipsoïde. Une vraie rotation 3D des points déchirait l'image : les points du bord de la tête passaient devant le visage, car une image 2D ne peut rien cacher.



Le centre bouge beaucoup

Les bords, le front et le menton, presque pas

Le modèle de la tête. Le mouvement diminue vers les bords, ce qui garde la silhouette en place.

$$x' = x + r \cdot (\sin(\operatorname{asin}(u) + \theta) - u)$$

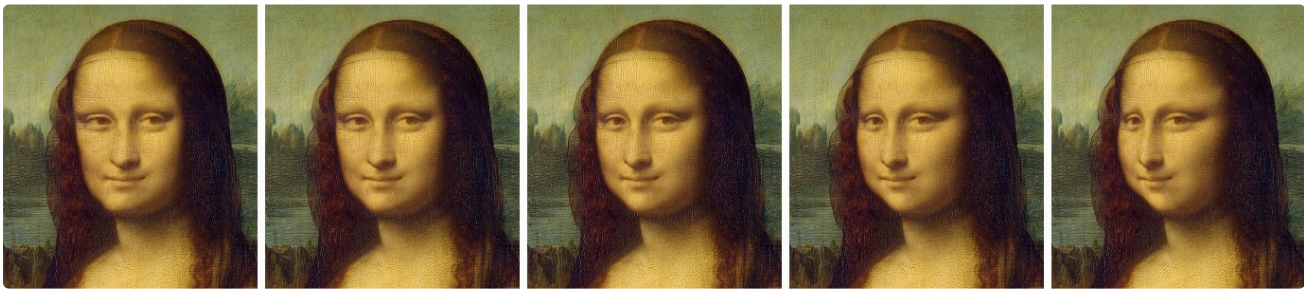
u = position du point relativement au rayon r · θ = angle donné au portrait · r diminue vers le haut et le bas

Le côté qui s'éloigne se comprime, et l'autre s'étire. L'ordre des points étant conservé, cela limite fortement les plis.

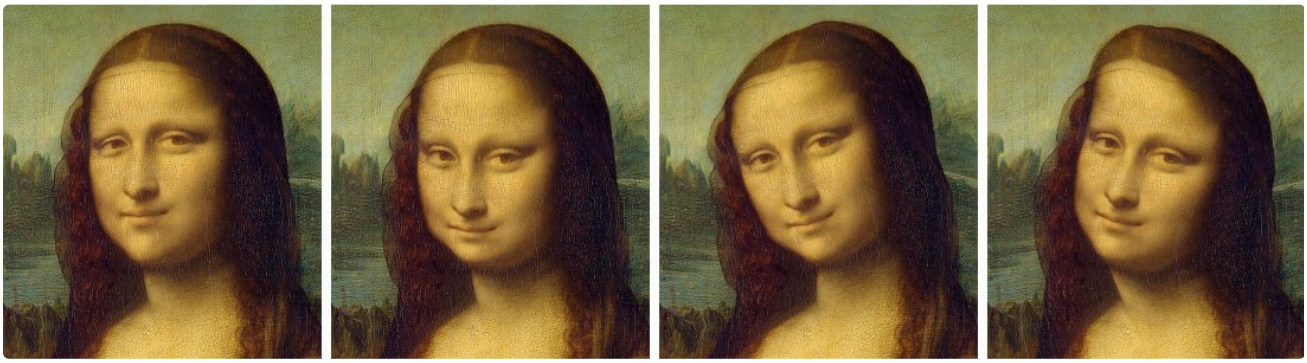
À ce modèle s'ajoute le **relief** : le nez et les lèvres, plus proches de la caméra, reçoivent un déplacement supplémentaire calculé avec z et les termes de R. Sur les portraits de trois quarts, une recherche par dichotomie (6 essais) trouve la plus grande part de ce relief qui ne crée aucun pli.

Rotation	Nom	Part transmise	Maximum sur le portrait
Gauche-droite	Lacet	60 %, atténuée	±14°
Haut-bas	Tangage	60 %, atténuée	±7°
Vers une épaule	Roulis	100 %	±15°

Pour le lacet et le tangage, une zone morte de 1,5° retire le bruit, et une tangente hyperbolique ralentit le mouvement à l'approche de la limite, au lieu de le bloquer d'un coup.



Une tête tournée de −30°, −12°, 0°, +12° et +30°. Le visage tourne, sans déchirure. Les cheveux et le fond ne bougent pas. Simulation.

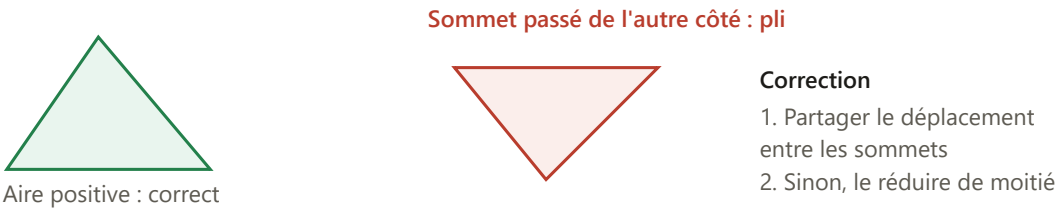


Tête levée, tête baissée, puis penchée vers chaque épaule. La rotation haut-bas reste plus discrète, car elle déforme vite le visage. Simulation.

> **C'est une illusion, et il faut le dire**
> Aucune nouvelle partie du visage n'apparaît : le portrait ne peut pas montrer une oreille cachée. Cette rotation n'est crédible qu'à faible angle. Cela explique pourquoi nous l'avons volontairement limitée.

4.10 Les garde-fous contre les plis

Un triangle se retourne quand ses sommets changent de sens : l'image s'y replie. On le détecte avec le signe de son aire, calculé par produit vectoriel. Si ce signe change, le triangle est retourné.



Détecter et corriger un pli. Trois sommets qui bougent ensemble forment une translation, qui ne peut rien replier.

Les petits plis de moins de 8 pixels carrés sont tolérés, car ils ne se voient pas. Les corriger rouvrait toute la paupière d'un œil en train de se fermer.

4.11 Peindre l'intérieur de la bouche

Le portrait a la bouche fermée : aucun pixel de l'intérieur n'existe. Étirer les lèvres donne des rayures, comme le montre la figure. Le sujet proposait de peindre cette zone avant la déformation, mais une bouche fermée n'a presque aucune surface à étirer. Nous peignons donc **après** la déformation, dans le polygone formé par les lèvres intérieures.



Sans et avec l'intérieur peint. Les triangles entre les lèvres étirent quelques pixels et laissent des rayures, que la peinture recouvre. Simulation.

- **L'ombre :** une transformée de distance indique l'éloignement de chaque pixel par rapport au bord. Plus on s'éloigne des lèvres, plus la couleur s'assombrit, comme en profondeur.
- **La couleur :** on garde la luminosité des lèvres, avec une teinte rouge imposée. En gardant leur teinte, les lèvres ocre de la Joconde donnaient du marron.
- **Les dents :** une bande courte sous la lèvre, couleur ivoire. Leur clarté suit celle de la peau : un blanc pur trancherait sur un tableau sombre.
- **La langue :** une ellipse floue et discrète. Une première version trop visible ressemblait à une boule rouge.
- **L'apparition progressive :** l'ouverture est mesurée sur la vraie bouche filmée, en proportion de l'écart entre les yeux. L'intérieur apparaît entre 2 et 5,5 %, les dents entre 8 et 14 %, et la langue entre 14 et 26 %.

- **Le fondu :** des filtres gaussiens adoucissent les bords, pour que la peinture se fonde dans le tableau.

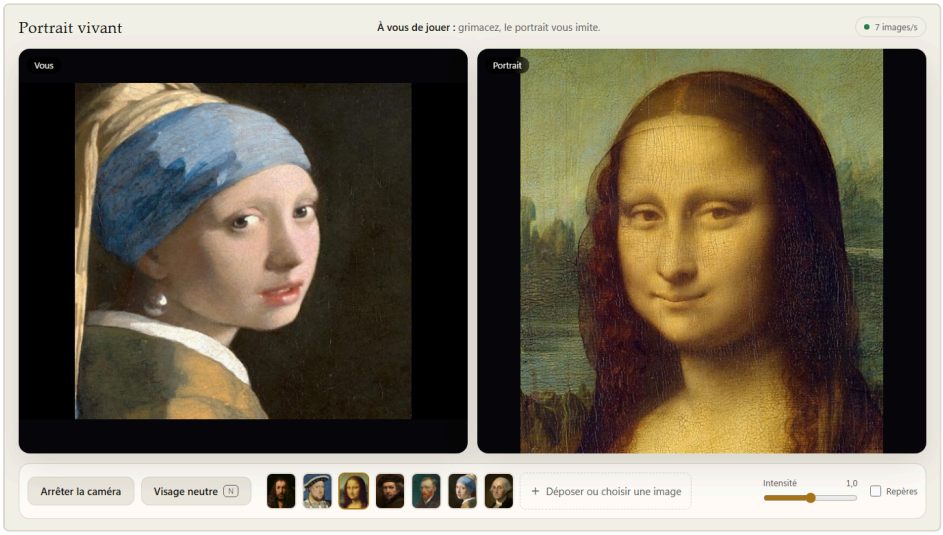
> **Pourquoi ne pas mesurer l'ouverture sur le portrait ?**
> L'échelle et les gains amplifient tout. Sur une webcam où les coins des yeux sont à 80 px, une lèvre qui bouge de 1 px devient 2,6 px sur la Joconde. Avec l'ancienne règle, cela affichait 53 % du noir : un trou apparaissait alors que la bouche restait fermée.



La bouche s'ouvre progressivement. L'ombre, puis les dents, et enfin une langue à peine visible apparaissent. Simulation.

4.12 L'interface

L'interface est une page web locale, ouverte dans le navigateur et servie par Python. Elle permet de déposer une image, de choisir un portrait et d'activer la caméra, sans passer par le terminal ni par le dossier des portraits.



L'interface, avec une fausse caméra. La personne filmée est ici un tableau au repos : la bouche du portrait reste fermée. Le rythme affiché n'est pas représentatif : cet essai ajoutait une attente artificielle, et le navigateur tournait en mode capture.

- **Deux parties :** un moteur tourne la boucle webcam dans un fil d'exécution séparé, et un petit serveur répond à la page. Les images sont envoyées sous forme de suite d'images JPEG (2,4 ms d'encodage par image pour les deux flux).
- **Sécurité :** le serveur n'écoute que sur 127.0.0.1 et refuse les demandes provenant d'autres sites, qui pourraient sinon viser la webcam.
- **Vie privée :** aucune image ne quitte l'ordinateur. Une image déposée reste en mémoire jusqu'à la fermeture du programme.
- **Webcam :** elle est vraiment libérée quand on clique sur « Arrêter la caméra ».
- **Fluidité :** les calculs se font hors du verrou partagé, sinon la page se figeait dès que la caméra tournait (voir section 7).

5. Le lien avec le cours

Le projet repose surtout sur la transformation affine et les moindres carrés ; plusieurs notions de la liste du sujet ne sont pas utilisées, et il vaut mieux le dire avant que le jury ne le demande. Cette liste vient du sujet : vérifiez le vocabulaire exact avec vos supports de cours.

Notion	État	Où, et comment l'expliquer
Affine à 3 points	Utilisée, cœur	`deformer_triangle` : 236 fois par image, 6 inconnues et 6 équations
Moindres carrés	Utilisés	`similitude_3d_moindres_carres` : la meilleure superposition des points stables
Translation, rotation, échelle	Utilisées	La similitude du recalage et la rotation du portrait dans le plan
RANSAC	Partiel	Seulement dans le recalage 2D de comparaison. Le programme fait un simple rejet
Points d'intérêt, appariement	Simplifiés	MediaPipe donne les points, appariés par leur numéro, sans descripteur
Filtre gaussien	Utilisé	Bords de la bouche, dents et langue
Filtrage temporel	Utilisé	Filtre exponentiel sur les repères et les angles
R et t, projection	Partiels	R et t au recalage, R à la rotation. Projection simplifiée, sans modèle sténopé complet
Couleur et luminosité	Utilisées	BGR et RVB, luminosité des lèvres et de la peau, teinte de la bouche
Interpolation	Utilisée	Bilinéaire dans `warpAffine`, et `INTER_AREA` à la réduction d'image
Homographie à 4 points	Non utilisée	Un triangle n'a que 3 points : l'affine suffit. Un essai de redressement a été retiré
Filtres médian et bilatéral	Non utilisés	Une médiane sert à estimer une couleur, sans filtrer l'image
K, distorsion, calibration de Zhang	Non utilisés	Bonus du sujet, non réalisé

Au-delà de la liste du sujet

Trois méthodes ne figurent pas dans la liste, et il faut savoir les expliquer : la décomposition en valeurs singulières (Umeyama), la transformée de distance et le modèle de tête en ellipsoïde.

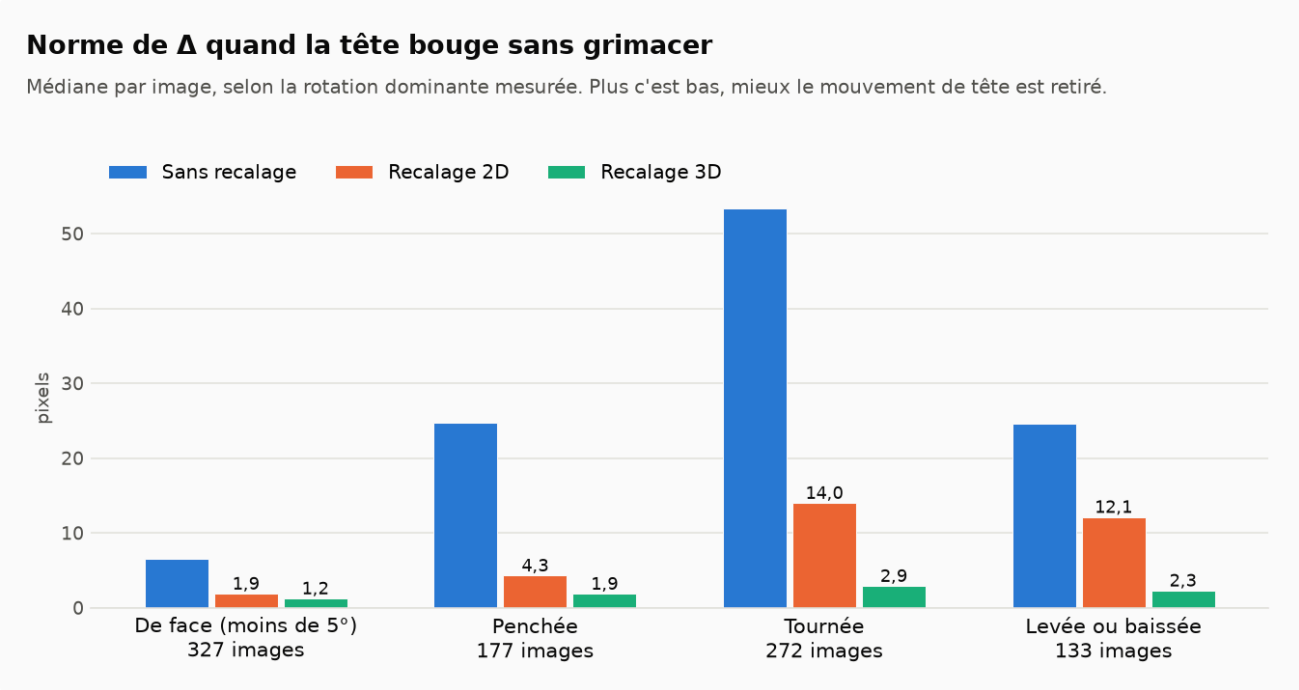
6. Les résultats et leur lecture

Le programme tient 24 images/s sans GPU, et le recalage 3D est nettement meilleur que le 2D dès que la tête tourne ou se lève. Ces mesures ont été faites avec une vraie webcam, sur une session et une personne, avant l'interface et la nouvelle bouche. Relancez `python mesures.py tout` sur la version finale.

6.1 À quoi servent ces mesures ?

Mesure	Question posée	Ce qu'elle permet de dire
Temps	Le programme tourne-t-il en direct sans GPU ?	Oui, et on sait quelle étape coûte le plus
Recalage	Un mouvement de tête est-il pris pour une grimace ?	Justifie le passage du 2D au 3D
Lissage	Le filtre réduit-il le tremblement ?	Oui, avec un gain modeste
Détection	Sur quels portraits la méthode marche-t-elle ?	Montre où elle s'arrête

6.2 Le recalage : le résultat principal



Sans grimace, Δ devrait être nul : tout ce qui reste est une erreur. 909 images classées selon la rotation réellement mesurée. Source : `resultats/recalage_resume.csv`.

- **Sans recalage,** l'erreur atteint 53,3 px quand la tête tourne : le portrait grimacerait sans cesse.
- **En 2D,** l'erreur reste faible quand la tête est penchée (4,3 px), car une similitude 2D sait corriger une rotation dans le plan. Elle monte à 14,0 px quand la tête tourne et à 12,1 px quand elle se lève ou se baisse.
- **En 3D,** l'erreur reste entre 1,2 et 2,9 px dans tous les cas, avec environ 5 fois moins d'erreur que le 2D sur les rotations hors du plan.
- **Pics :** le 2D dépasse 30 px de norme moyenne sur 27 images, le 3D jamais.

> **Précaution de lecture**
> Les angles viennent du recalage 3D lui-même, sans appareil de référence. Ce résultat mesure donc la cohérence de la méthode, pas sa précision absolue. Les valeurs sont en pixels de webcam : comparez les méthodes entre elles, dans une même exécution.

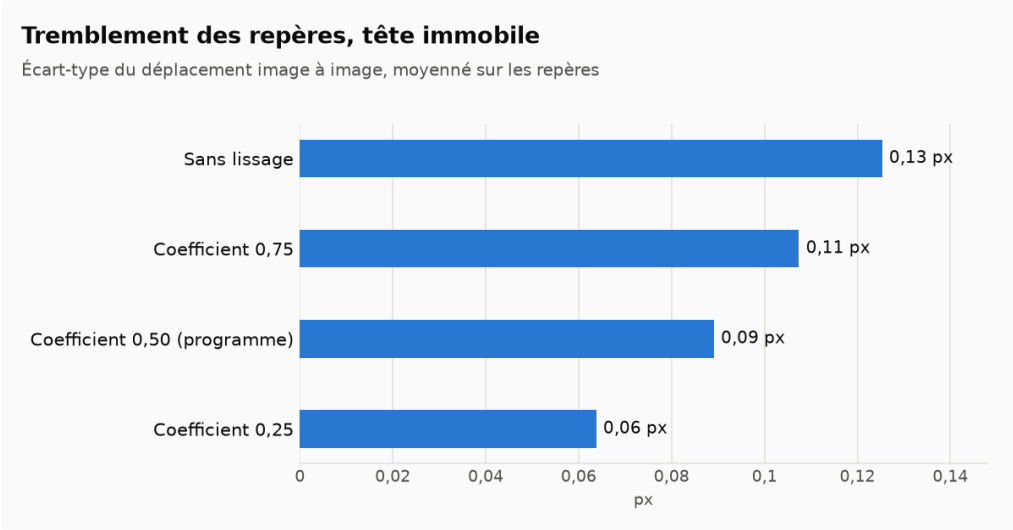
6.3 La vitesse

Étape	ms	Part
Déformation	21,6	53 %
Détection	9,4	23 %
Affichage	6,2	15 %
Lecture webcam	2,5	6 %
Transfert	0,8	2 %
Lissage et recalage	0,4	1 %
Image complète	**41,0**	

24,4 images/s sur 300 images, dans la fenêtre OpenCV, avec un Intel Core i7-1165G7 sans GPU.

La déformation prend plus de la moitié du temps : c'est elle qu'il faudrait accélérer en premier. Le recalage et le transfert, eux, ne coûtent presque rien : la géométrie est peu coûteuse par rapport aux images à manipuler.

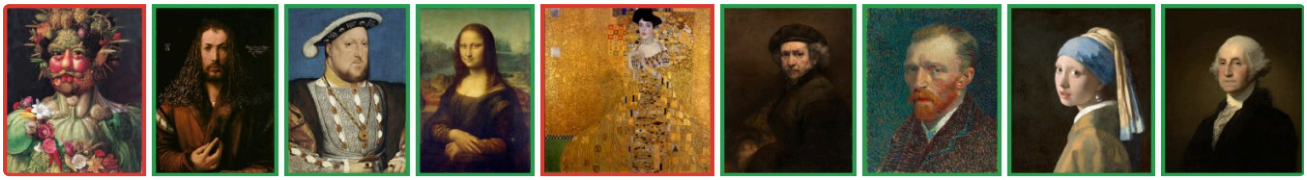
6.4 Le lissage



Tremblement des repères, tête immobile. Il passe de 0,125 px à 0,089 px avec le coefficient du programme (−29 %), et à 0,064 px avec 0,25 (−49 %). Source : `resultats/lissage.csv`.

Les repères sont déjà très stables, donc le lissage aide sans être décisif. Son coût, le retard, n'a pas été mesuré : c'est une limite à signaler.

6.5 La détection



7 portraits détectés sur 9 (cadre vert : détecté · cadre rouge : non détecté). De gauche à droite : Arcimboldo, Dürer, Holbein, Joconde, Klimt, Rembrandt, Van Gogh, Vermeer, Washington.

La détection échoue sur Arcimboldo, dont le visage est fait de fruits, et sur Klimt, où le visage est très petit dans une grande image. Comme ces œuvres ont été choisies à la main, ce n'est pas un taux de réussite général.

6.6 Comment les mesures sont faites

Chaque mesure suit les consignes affichées à l'écran. L'ensemble dure environ 1 min 30 avec `python mesures.py tout`.

Mesure	Ce que fait la personne	Ce qui est calculé
Lissage	Visage détendu, puis ne plus bouger pendant 10 s	L'écart-type du déplacement image à image, pour plusieurs coefficients, sur la même série de points
Recalage	Cinq mouvements de 6 s sans grimacer : déplacer, approcher, pencher, tourner, lever	La norme de Δ à chaque image, sans recalage, en 2D et en 3D, puis la médiane par rotation mesurée
Temps	Grimaces normales pendant 300 images	La durée moyenne de chaque étape, et le nombre d'images par seconde
Détection	Rien : aucune webcam n'est nécessaire	Un visage est-il trouvé sur chaque portrait ?

Pour le recalage, chaque image est classée selon sa rotation dominante mesurée, et non selon la consigne affichée : lors de notre session, les mouvements ne correspondaient pas aux consignes. Une pause de 2 s a depuis été ajoutée entre les mouvements. Par conséquent, ce classement ne sépare pas la translation et la distance.

6.7 Le message à retenir

> **À dire à l'oral**
> « Le programme tient 24 images par seconde sans GPU. Le recalage est indispensable, et notre passage au 3D, qui n'était pas dans le sujet, divise l'erreur par environ 5 quand la tête tourne. En revanche, ces chiffres viennent d'une seule session et mesurent la cohérence de la méthode, pas sa précision absolue. »

Comment présenter les figures

- **Les images simulées** de ce dossier expliquent le principe. Précisez-le si vous les réutilisez.
- **Les graphiques de** `resultats/` viennent de vraies mesures, à refaire sur la version finale.
- **La qualité visuelle** ne se mesure pas ici : la démo et des captures réelles la montrent.

7. Les problèmes rencontrés et nos décisions

La plupart de nos choix viennent d'un problème observé, pas d'une idée de départ. Cette section est utile à l'oral : raconter un problème, sa cause et sa solution montre que vous avez compris le projet.

Problème observé	Cause	Solution
Tourner la tête déformait tout le portrait	Une similitude 2D ne corrige pas une rotation 3D	Recalage 3D avec la profondeur de MediaPipe
Le portrait restait ensuite figé quand la tête tournait	Le recalage 3D retirait la rotation, sans la transférer	Transfert contrôlé de la rotation
Déchirure en zigzag sur le côté du visage	En vraie rotation, les points du bord passaient devant le visage	Silhouette gardée en place, puis modèle en ellipsoïde
Mouvement de tête peu naturel	Trop d'amplitude, arrêt brutal, bruit amplifié, front et menton trop mobiles	60 % transmis, limite progressive, lissage dédié, ellipsoïde
Clin d'œil trop faible	Un déplacement en pixels ne s'adapte pas à la taille de l'œil	Transfert d'une proportion d'ouverture
Le portrait ne suivait pas le regard	L'iris bouge de quelques pixels seulement	Position relative, moyenne des deux yeux, gain de 2,5
Sourire trop faible	Le garde-fou annulait le mouvement à cause d'un minuscule triangle (22 px ramenés à 5,5 px en simulation)	Partage du mouvement entre voisins, gains, points sur les joues
Plis à chaque clignement	Le haut et le bas de l'iris sortent des paupières	Seul le centre de l'iris est gardé
Plis sur Vermeer et Holbein	De trois quarts, un point du contour tombe dans l'œil	Ces points ne deviennent plus des sommets
Rayures dans la bouche ouverte	Une bouche fermée n'a aucune surface à étirer	Intérieur peint après la déformation
Trou sombre, bouche fermée	L'ouverture était mesurée sur le portrait, où l'échelle et les gains amplifient le moindre mouvement de lèvre	Ouverture mesurée sur la vraie bouche, en proportion du visage
Bouche marron, puis « dessin animé »	La teinte ocre des lèvres était conservée, puis les dents et la langue étaient trop visibles	Teinte rouge imposée, éléments atténués et fondus
Page figée dès que la caméra tournait	Le moteur gardait son verrou pendant presque tout le calcul	Calcul sur une copie de l'état, hors du verrou
Graphique du recalage trompeur	Les mouvements ne suivaient pas les consignes : 6 s par mouvement, c'était trop court	Images classées par angle mesuré, et pause entre les mouvements

Les écarts avec le sujet

Le sujet demandait	La version finale
Recalage 2D avec RANSAC	Recalage 3D, le 2D étant gardé pour la comparaison
Rotation du portrait dans le plan, ±15°	Roulis ±15°, plus lacet ±14° et tangage ±7°
Bouche peinte avant la déformation	Bouche peinte après la déformation
Masque anticrénelé	Masque net sur 8 bits
Tout l'iris dans les repères	Le centre de l'iris seulement
Fenêtre OpenCV	Fenêtre conservée, avec une interface web en plus
Placement manuel si aucun visage n'est détecté	**Retiré.** Un portrait sans visage détecté est ignoré, avec un message

À préparer

Le placement manuel, un vrai RANSAC 3D, l'homographie de redressement et les filtres médian et bilatéral ont été développés, puis retirés par décision de l'équipe. Le placement manuel étant une exigence du sujet, préparez la raison de ce choix : elle n'est pas écrite dans ce dossier.

Comment raconter un problème à l'oral

Utilisez toujours la même structure en quatre temps : ce qu'on a vu, pourquoi, ce qu'on a fait, et comment on l'a vérifié. Elle tient en quelques phrases et montre votre démarche.

Temps	Exemple avec la tête qui tourne
Ce qu'on a vu	« Quand je tournais la tête sans grimacer, le visage du portrait se déformait complètement. »
Pourquoi	« Notre recalage était une similitude 2D, alors que tourner la tête est une rotation en 3D : le nez bouge plus que les joues. »
Ce qu'on a fait	« Nous avons utilisé la profondeur donnée par MediaPipe pour estimer une rotation en 3D, par moindres carrés. »
Comment on l'a vérifié	« Quand la tête tourne, l'erreur passe de 14 px à 2,9 px sur nos mesures. »

Trois exemples à préparer

Choisissez chacun un problème du tableau : la rotation de la tête, le sourire annulé par le garde-fou, et l'intérieur de la bouche sont les plus parlants.

8. Les limites à assumer

La méthode est géométrique : elle déplace les pixels existants et n'invente pas ce que l'image ne montre pas. Annoncer ces limites vous-même montre que vous maîtrisez le sujet.

Limite	Ce qu'on observe	Piste possible
Rotation de la tête	Réduite à de petits angles, le bord du visage se déforme à grand angle. Aucune partie cachée n'apparaît	Un modèle génératif, mais avec plus de calcul
Portraits de trois quarts	Tournent moins d'un côté. Le sourire tombe à 55-65 % d'un côté sur Holbein et Van Gogh	Modèle de visage plus fin
Détection	Échoue sur Arcimboldo et Klimt, et ces œuvres ne sont alors pas utilisables	Placement manuel, recadrage avant détection
Bouche	L'intérieur est peint : c'est une apparence plausible, pas la vraie bouche	Génération d'images
Œil fermé	Peut garder une ligne sombre un peu dure, par exemple sur Vermeer	Peindre aussi la paupière
Déformation	Prend 53 % du temps de calcul	Ne recalculer que les triangles qui ont bougé
Mesures	Une personne, une session, sans vérité terrain, et le retard n'est pas mesuré	Refaire les mesures à trois, et mesurer le retard
Interface	La fluidité et le glisser-déposer restent à confirmer avec la vraie webcam	Essai complet avant la soutenance

9. Préparer la présentation et la démo

Commencez par la démo : elle fait comprendre le projet en quelques secondes. Adaptez les durées au temps imposé et répartissez les parties entre vous trois.

#	Partie	Contenu	Part
1	**Accroche**	Une grimace en direct : la Joconde l'imite, puis un clin d'œil et un sourire	10 %
2	**L'idée**	La phrase clé, puis le schéma de la section 3	10 %
3	**Le cœur du cours**	3 points, 6 inconnues, Delaunay. Montrez les triangles avec la case « Repères »	20 %
4	**Le recalage**	La figure de la déchirure, le passage au 3D, puis le graphique	25 %
5	**Ce qui n'existe pas**	La bouche sans et avec peinture	10 %
6	**Les résultats**	24 images/s, et ce que disent les mesures	10 %
7	**Limites et suites**	Annoncez-les vous-mêmes, puis concluez	15 %

Les phrases utiles

- « On ne copie pas le visage : on transfère le mouvement. »
- « Chaque pixel déplacé s'explique par une transformation affine. »
- « Sans grimace, le déplacement devrait être nul : ce qui reste, c'est notre erreur. »
- « Notre méthode est explicable et tourne sans GPU, mais elle ne peut pas inventer ce que l'image ne montre pas. »

Pendant la démo

- Lancez `python interface.py` avant de commencer, et gardez la page ouverte.
- Cliquez sur « Visage neutre » visage détendu, sinon toutes les grimaces seront faussées.
- Faites face à la caméra, avec une lumière sur le visage.
- Choisissez la Joconde, Dürer ou Rembrandt, plus de face. Évitez Vermeer pour la rotation.
- Tournez lentement et sans excès : la limite se voit à grand angle.
- Pour un effet garanti : clin d'œil, sourire, puis bouche grande ouverte.

À préparer avant le jour J

☐	Essayer l'interface avec la vraie webcam : image, rythme, boutons et glisser-déposer
☐	Relancer `python mesures.py tout` sur la version finale, idéalement sur vous trois
☐	Enregistrer une vidéo de la démo : c'est votre plan B
☐	Préparer des captures avant/après pour les diapositives
☐	Tester sur l'ordinateur et, si possible, dans la salle de la présentation
☐	Vérifier que `INDICE_CAMERA` dans `main.py` désigne la bonne caméra
☐	Préparer la raison du retrait du placement manuel
☐	Décider qui répond à quels thèmes de questions, et qui a fait quoi
☐	Vérifier les règles de votre formation sur l'usage d'une IA pour programmer

> **Si la démo plante**
> Ne cherchez pas à réparer devant le jury. Dites : « Nous avons prévu une vidéo enregistrée », lancez-la et continuez. Un plan B prêt à l'emploi fait une bonne impression.

Le déroulé conseillé de la démo

1. **Avant de parler :** l'interface est ouverte, la caméra éteinte et la Joconde sélectionnée.
2. **Activer la caméra :** dites que tout reste sur l'ordinateur.
3. **Visage neutre :** expliquez en une phrase pourquoi ce visage de référence est nécessaire.
4. **Trois grimaces :** un clin d'œil, un sourire, puis la bouche grande ouverte.
5. **Montrer la mécanique :** cochez « Repères » pour faire apparaître les points et les triangles, puis décochez.
6. **Changer de portrait :** cliquez sur une autre miniature, sans refaire le visage neutre.
7. **Montrer la limite :** tournez la tête franchement, et expliquez pourquoi cela se dégrade.

Les erreurs à éviter

- Prendre le visage neutre en souriant ou la tête de côté.
- Dire que le programme utilise RANSAC, ou qu'il est « en 3D » : seul le recalage l'est, et l'image reste plate.
- Annoncer une comparaison avec LivePortrait qui n'a pas été faite.
- Présenter une image simulée comme un résultat réel.
- Promettre un rythme de 24 images/s dans l'interface sans l'avoir mesuré.

10. Les questions probables du jury

Ces réponses sont courtes, pour être dites à l'oral. Reformulez-les avec vos mots : le jury verra vite si une réponse est apprise sans être comprise. Si vous ne savez pas, dites-le, puis expliquez ce que vous vérifieriez.

Le principe

Quelle est l'idée du projet, en une phrase ?

Mesurer comment mes points bougent par rapport à mon visage au repos, appliquer ce mouvement aux points du portrait, puis déformer l'image triangle par triangle.

Pourquoi transférer un déplacement, et pas les positions ?

Mon visage et le portrait n'ont pas la même forme. Copier mes positions lui donnerait ma tête. Δ ne contient que la grimace : on l'ajoute à ses points.

À quoi sert le visage neutre ?

C'est la référence : Δ se calcule par rapport à lui. Sans lui, on ne sait pas ce qui a changé. S'il est pris en pleine grimace, tout est décalé.

Pourquoi MediaPipe, qui n'est pas dans le cours ?

Le sujet l'impose pour détecter les repères. Il fournit les points, et nous programmons toute la géométrie qui suit. C'est la seule partie apprise du programme.

La déformation et les triangles

Pourquoi des triangles ?

Trois points déterminent exactement une transformation affine : 6 inconnues, 6 équations. Et deux triangles voisins partagent une arête, donc l'image reste continue.

Quelle différence entre une affine et une homographie ?

L'affine a 6 inconnues et garde le parallélisme. L'homographie en a 8, demande 4 points et modélise la perspective. Un triangle n'ayant que 3 points, et les déformations locales étant petites, l'affine suffit.

Pourquoi Delaunay ? Pourquoi une seule fois ?

Elle évite les triangles très fins : le cercle circonscrit de chaque triangle ne contient aucun autre point. On la calcule une fois, car une triangulation qui change à chaque image ferait sauter l'image.

Pourquoi repartir de l'image d'origine ?

Chaque interpolation floute un peu. Repartir du résultat précédent accumulerait ce flou et les erreurs.

Pourquoi 8 points sur le bord ?

Ils restent fixes et ferment la triangulation : les triangles couvrent toute l'image, et le cadre ne bouge pas.

Pourquoi avoir changé le masque de notre fonction de départ ?

OpenCV ignore `LINE_AA` sur un masque `float32` : on l'a vérifié. Un masque 8 bits net donne le même rendu, sans joint, et a été 2,7 fois plus rapide dans notre essai.

Qu'est-ce qu'un triangle retourné ? Comment le gérez-vous ?

Ses sommets changent de sens, et l'image s'y replie. Le signe de son aire le révèle. On partage alors le déplacement entre ses sommets, puis on le réduit si besoin. Les plis de moins de 8 pixels carrés sont tolérés.

Pourquoi 117 points, et pas les 478 ?

Plus de points donnent plus de triangles à calculer, alors que la déformation prend déjà 53 % du temps. Ce sous-ensemble suffit à décrire les expressions, et le sujet demandait environ 80 à 120 points.

Que se passe-t-il si un triangle sort de l'image ou s'aplatit ?

Sa boîte est découpée aux bords de l'image. S'il n'a plus de surface, il est ignoré, car une transformation affine ne se calcule pas avec 3 points alignés. Des tests couvrent ces deux cas.

Le recalage

Pourquoi un recalage ?

Si je bouge la tête sans grimacer, mes points bougent quand même. Sans recalage, l'erreur atteint 53 px quand la tête tourne, et le portrait grimacerait sans raison.

Pourquoi le 2D ne suffit-il pas ?

Une similitude 2D corrige la translation, l'échelle et la rotation dans le plan. Tourner la tête est une rotation 3D : le nez bouge plus que les joues, ce qu'aucune similitude 2D ne peut compenser.

Expliquez le recalage 3D.

On cherche s, R et t qui superposent au mieux les 9 points stables sur le visage neutre, par moindres carrés. On centre les nuages de points, la SVD de leur covariance donne R, puis on déduit s et t.

Pourquoi seulement certains points ?

Coins des yeux, nez, tempes et front bougent peu quand on grimace. Avec la bouche, le calcul prendrait la grimace pour un mouvement de la tête.

Utilisez-vous RANSAC ?

Seulement dans le recalage 2D de comparaison. Le recalage 3D du programme fait un rejet simple : une estimation, le retrait des points éloignés, puis une nouvelle estimation. Un vrai RANSAC tire des échantillons au hasard et garde celui qui explique le plus de points.

La profondeur z de MediaPipe est-elle fiable ?

Ce n'est pas une vraie mesure de distance, mais elle suffit ici. Entre portraits orientés différemment de 15 à 19°, l'erreur passe de 6-13 % de l'écart entre les yeux en 2D à 2-4 % en 3D.

Pourquoi pas de calibration de la caméra ?

Le recalage compare les points de la même caméra entre eux, sans mesure métrique. Zhang avec `solvePnP` était un bonus, non réalisé. Cela donnerait une pose plus rigoureuse, sans résoudre le manque d'information dans le portrait.

Votre modèle de caméra est-il un sténopé ?

Pas complètement. On applique R aux points (x, y, z), puis on garde x et y, sans division par la profondeur ni matrice K. C'est une projection simplifiée, suffisante pour de petits mouvements.

Le transfert et le rendu

Comment adapter le mouvement à la taille du portrait ?
Avec s, l'écart entre les yeux du portrait divisé par le mien. Des gains renforcent les lèvres, les joues et les sourcils, car un mouvement fidèle paraît trop discret sur un tableau.

Pourquoi traiter les yeux à part ?
En pixels, l'œil du portrait ne se fermait pas entièrement. On transfère donc une proportion d'ouverture : sous 35 % de l'ouverture au repos, l'œil compte comme fermé.

Comment une tête tourne-t-elle dans une image plate ?
Elle ne tourne pas vraiment : c'est une illusion. On modélise la tête par un ellipsoïde, où le centre bouge beaucoup et les bords presque pas, et on ajoute le relief avec z. Aucune partie cachée n'apparaît, d'où la limite d'angle.

Comment est dessiné l'intérieur de la bouche ?
Après la déformation, dans le polygone des lèvres intérieures. Une transformée de distance assombrit le centre, puis on ajoute des dents et une langue discrètes. La clarté dépend du tableau, et un filtre gaussien fond les bords. Tout apparaît selon l'ouverture mesurée sur ma vraie bouche.

Comment éviter que la bouche s'ouvre à tort ?
On mesure l'ouverture sur la vraie bouche, tête remise en place, en proportion de l'écart entre les yeux. Une lèvre qui se détend reste sous le seuil de 2 %, et le portrait n'affiche aucun trou.

Que se passe-t-il si je sors du cadre ?
Le portrait garde son dernier état, et un message s'affiche. Au retour du visage, le filtre repart de zéro pour ne pas traîner une position ancienne.

Pourquoi un effet miroir ?
Pour que la démo soit naturelle : quand je ferme mon œil droit, l'œil qui se ferme se trouve du même côté de l'écran, comme dans une glace.

Pourquoi une page web pour l'interface ?
Elle permet facilement le glisser-déposer et un rendu moderne, sans dépendance supplémentaire. Tout reste local : le serveur n'écoute que sur cet ordinateur et refuse les demandes des autres sites.

Les résultats

Le temps réel est-il atteint ? Où passe le temps ?
Oui, à 24 images/s dans la fenêtre OpenCV, sans GPU. La déformation prend 53 % du temps, la détection 23 %. Il faudrait accélérer la déformation en premier.

Sans vérité terrain, que vaut votre mesure ?
Elle mesure la cohérence, pas la précision absolue. Sans grimace, Δ devrait être nul : son reste indique l'erreur. Les angles viennent du recalage lui-même, et nous n'avons qu'une session.

Pourquoi des médianes dans le graphique ?
Quelques images où une méthode décroche fausseraient une moyenne. La médiane montre le comportement habituel, et nous signalons les pics à part.

Le lissage est-il vraiment utile ?

Il réduit le tremblement de 29 %, mais les points étaient déjà très stables. Son prix est un retard, que nous n'avons pas mesuré.

Avez-vous testé votre programme ?

Oui, avec 171 tests automatiques, puis des essais à la webcam. Ces tests utilisent des images et des grimaces simulées : ils ne jugent ni la qualité visuelle ni la vraie webcam.

L'ouverture et les questions pièges

Pourquoi avoir retiré le placement manuel, demandé par le sujet ?

À préparer en équipe : donnez votre vraie raison. La conséquence à reconnaître : un portrait sans visage détecté est ignoré.

En quoi est-ce différent d'un modèle comme LivePortrait ?

Notre méthode tourne sans GPU, et chaque étape s'explique. Un tel modèle peut générer ce qui manque dans l'image, mais demande plus de calcul. Nous n'avons pas fait la comparaison.

Est-ce un deepfake ? Quels sont les risques ?

C'est une animation de portrait, et le rendu reste visiblement déformé. Le risque existe pour une photo de personne réelle : le projet se limite donc au domaine public et aux personnes d'accord. Rien ne quitte l'ordinateur.

Avez-vous utilisé une IA pour programmer ?

Répondez honnêtement : le code a été développé avec l'assistant Claude Code. Vérifiez les règles de votre formation, et assurez-vous que chacun sait expliquer le code qu'il présente.

Que feriez-vous avec plus de temps ?

Accélérer la déformation, réintroduire le placement manuel, faire un vrai RANSAC 3D, tester sur plus de personnes, et comparer avec un modèle génératif.

Quelle est la plus grande difficulté rencontrée ?

La rotation de la tête : plusieurs versions, de la similitude 2D à l'ellipsoïde. Cela nous a appris qu'une image n'est pas un objet 3D et qu'un bon compromis vaut mieux qu'une méthode exacte mais inutilisable.

Que vous a appris ce projet ?

À préparer avec vos mots. Une piste : le passage d'une formule simple à un programme robuste demande surtout de gérer les cas limites, comme les plis, les visages de trois quarts ou ce que l'image ne contient pas.

Comment répondre quand on ne sait pas

> **Une réponse honnête vaut mieux qu'une invention**
>
> Dites par exemple : « Je ne l'ai pas mesuré. Pour le vérifier, je ferais tel essai. » Cette réponse montre votre rigueur. Reformulez aussi la question si elle n'est pas claire, et n'hésitez pas à passer la parole à la personne de l'équipe qui connaît le mieux la partie concernée.

Les chiffres à connaître par cœur

Chiffre	Ce qu'il mesure
24 images/s	Rythme mesuré dans la fenêtre OpenCV, sans GPU
117 + 8 points	Les repères du visage, plus les points fixes du bord
236 triangles	Sur la Joconde : une transformation affine pour chacun
6 inconnues	Une affine, déterminée par 3 points. L'homographie en a 8, avec 4 points
14 → 2,9 px	L'erreur de recalage en 2D puis en 3D, quand la tête tourne
53 %	La part du temps prise par la déformation
−29 %	La baisse du tremblement grâce au lissage
7 sur 9	Les portraits détectés parmi nos essais

Annexes

A. Le plan du code

Fichier	Rôle	Fonctions à connaître
`reperes.py`	Détection, 117 repères, lissage	`DetecteurReperes`, `LissageExponentiel`
`recalage.py`	Retrait du mouvement, calcul de Δ	`calculer_grimace_3d`, `similitude_3d_moindres_carres`
`transfert.py`	De Δ aux points du portrait	`transferer`, `appliquer_pose`, `empecher_retournements`
`deformation.py`	Portrait, triangles, bouche	`deformer_triangle`, `trianguler`, `peindre_bouche`
`main.py`	Version en fenêtre OpenCV	`animer_le_portrait`, `executer`
`interface.py`, `.html`	Interface locale	`Moteur`, `Gestionnaire`
`mesures.py`	Mesures, CSV et graphiques	`comparer_recalages`, `mesure_temps`
`tests/`	171 tests automatiques	`test_*.py`

B. Les commandes

Depuis le dossier `portrait_vivant`, avec l'environnement activé (sinon, utilisez `.\.venv\Scripts\python.exe`) :

`python interface.py`	Lancer l'interface dans le navigateur
`python main.py`	Lancer la version en fenêtre OpenCV
`python mesures.py tout`	Refaire toutes les mesures
`python mesures.py graphiques`	Retracer les graphiques
`python -m pytest tests`	Lancer les tests automatiques
`python tests/verifier_installation.py --webcam`	Vérifier l'installation

Touches : N (visage neutre), O (choisir une image), P (portrait suivant), + ou = et − (intensité), D (repères), Q (quitter, fenêtre OpenCV).

C. Les paramètres importants

Paramètre	Valeur	Effet
`COEFFICIENT_LISSAGE` (main)	0,5	Plus petit : plus stable, avec plus de retard
`COEFFICIENT_LISSAGE_POSE` (main)	0,25	Lissage des angles de la tête

Paramètre	Valeur	Effet
`GAIN_POSE` (transfert)	0,6	Part de la rotation transmise ; 0 retire lacet et tangage
`LACET_MAX`, `TANGAGE_MAX`, `ROULIS_MAX`	14°, 7°, 15°	Limites de rotation du portrait
`OUVERTURE_FERMEE`	0,35	Ouverture relative où l'œil compte comme fermé
`GAIN_REGARD`	2,5	Amplification du mouvement des pupilles
`GAINS`	1,5 et 1,3	Lèvres 1,5 ; joues et sourcils 1,3
`LIMITE_DEPLACEMENT`	0,8	Déplacement maximal, en écarts entre les yeux
`AIRE_REPLI_TOLEREE`	8 px^2	Taille des plis ignorés
`HAUTEUR_PORTRAIT` (deformation)	600 px	Plus grand : plus net, mais plus lent
`INDICE_CAMERA` (main)	1	Numéro de la webcam utilisée

D. Le lexique

Repère	Un point localisé sur le visage, comme le coin d'un œil
Visage neutre (M_0)	Les repères enregistrés au repos, qui servent de référence
Δ (delta)	Le déplacement de chaque repère par rapport au visage neutre : la grimace
Transformation affine	Transformation qui garde les droites et le parallélisme, définie par 3 points
Homographie	Transformation qui modélise la perspective, définie par 4 points
Similitude	Une rotation, une échelle et une translation, sans autre déformation
Moindres carrés	Chercher la solution dont la somme des erreurs au carré est la plus petite
RANSAC	Essayer des échantillons au hasard et garder celui qui explique le plus de points
SVD	Décomposition en valeurs singulières : une factorisation de matrice, qui donne ici la rotation
Delaunay	Relier des points en triangles, avec des cercles circonscrits vides
Recalage	Remettre le visage dans sa pose de référence pour comparer ses points
Lacet, tangage, roulis	Tourner la tête à gauche ou à droite, la lever ou la baisser, la pencher vers une épaule
Filtre exponentiel	Moyenne de la nouvelle mesure et de la valeur précédente, qui réduit le tremblement
Filtre gaussien	Moyenne pondérée des pixels voisins, qui adoucit l'image
Transformée de distance	Donne, pour chaque pixel, sa distance au bord d'une forme
Zone morte	Plage de petites valeurs ramenées à zéro, pour ignorer le bruit

E. Les portraits et les sources

Œuvres du domaine public, récupérées sur Wikimedia Commons : Léonard de Vinci, *La Joconde* · Johannes Vermeer, *La Jeune Fille à la perle* · Vincent van Gogh, *Autoportrait* · Albrecht Dürer, *Autoportrait* · Hans Holbein le Jeune, *Henri VIII* · Rembrandt, *Autoportrait* · Gilbert Stuart, *George Washington* · Giuseppe Arcimboldo, *Vertumne* · Gustav Klimt, *Adele Bloch-Bauer I*.

Documentation : guide de MediaPipe Face Landmarker pour Python. La méthode d'Umeyama vient de : S. Umeyama, *Least-squares estimation of transformation parameters between two point patterns*, IEEE TPAMI, 1991.